Combinatorial Purged Cross-Validation for Financial Machine Learning

Implementation, Validation, and Empirical Analysis

SPY 2024 — Three Scenarios

Based on:

Advances in Financial Machine Learning

Marcos López de Prado (2018), Chapters 7 & 11–12

April 8, 2026

Contents

1 Introduction and Motivation **2**

2 Data and Features **2**
2.1 Dataset . . . 2
2.2 Features . . . 2
2.3 Label and t_1 . . . 3
2.4 Model . . . 3

3 Implementation **3**
3.1 Purging — `getTrainTimes` (verbatim De Prado) . . . 3
3.2 Embargo — `getEmbargoTimes` (verbatim De Prado) . . . 3
3.3 Combinatorial Purged KFold . . . 4
3.4 Path Construction . . . 4
3.5 Performance Metrics . . . 4

4 Method Comparison: Nine Variants **5**

5 Scenario A: SPY 2024 (Clean) **5**
5.1 CPCV Split Structure . . . 5
5.2 CPCV Metrics by Split . . . 6
5.3 CPCV Path Results . . . 7
5.4 Method Comparison (Scenario A) . . . 8

6 Scenario B: SPY 2024 with Synthetic Crash (-20%) **9**

7 OOS Degradation and Probability of Backtest Overfitting **11**
7.1 IS vs OOS Sharpe Degradation . . . 11
7.2 Rank Logits and PBO . . . 11

8 Scenario C: Feature Leakage Detection **12**
8.1 Setup . . . 12
8.2 Results . . . 12
8.3 Interpretation . . . 13

9 OOS Sharpe Distributions by Validation Family **13**

10 Model Validation Map: Performance vs Overfitting Bias **14**

11 Walk-Forward Purge Debug **14**

12 Walk-Forward vs CPCV: Key Differences **15**

13 Summary and Conclusions **15**

1 Introduction and Motivation

Standard k-fold cross-validation (CV) assumes observations are independently and identically distributed (*i.i.d.*). Financial time series violate this assumption in two fundamental ways:

1. **Serial overlap.** When labels are constructed from overlapping windows (e.g. a 5-day forward return uses the same price bars across adjacent observations), training and test sets share information even when split at random.
2. **Temporal leakage.** Without an explicit time gap between train and test, a model trained on observations immediately before the test fold may have effectively "seen" future information through the overlapping labels.

López de Prado (2018) proposes two remedies and builds upon them to create a principled backtesting framework:

Purging Remove from the training set all observations whose label window overlaps with the test set (Snippet 7.1).

Embargo After purging, additionally exclude a short buffer of observations immediately following the test set to guard against auto-correlation in features (Snippet 7.2).

CPCV Combinatorial Purged Cross-Validation generates $\binom{N}{k}$ splits by choosing k of N time groups as the test set in each split. This produces $\phi = \binom{N}{k} k/N$ independent *paths* — complete simulated backtests — which form the basis for estimating the Probability of Backtest Overfitting (PBO).

This document describes the full implementation, reports results across three empirical scenarios, and draws conclusions about overfitting risk and leakage detection.

2 Data and Features

2.1 Dataset

We use SPY daily OHLCV data from 2024-01-01 to 2025-01-01, downloaded via `yfinance` (auto-adjusted closes). After removing NaN rows from feature construction, the final dataset contains $n = 227$ observations spanning 2024-01-31 to 2024-12-23.

The class balance for the binary label is 64.3% positive (up days), which reflects the bullish SPY trend in 2024.

2.2 Features

Six features are constructed from prices:

Feature	Formula	Intuition
`rsi_14`	RSI(14)	Momentum / mean-reversion signal
`momentum_20`	C_t/C_{t-20}	Medium-term trend
`rvol_20`	$\sigma(\log C_t/C_{t-1}, 20)$	Realized volatility
`ret_lag1`	$\log(C_t/C_{t-1})$, shifted by 1	Lagged return
`ret_lag5`	5-day rolling log return, shifted 1	Short-term momentum
`vol_norm`	$V_t/\overline{V}_{20}$	Volume anomaly

2.3 Label and t_1

The binary label is

$$y_t = \mathbf{1}\left[\log\left(\frac{C_{t+5}}{C_t}\right) > 0\right]$$

and the label end date $t_1(t) =$ index of bar $t+5$. The forward 5-day log-return $r_t^{\text{fwd}} = \log(C_{t+5}/C_t)$ is stored separately for economic PnL calculation.

2.4 Model

We use XGBoost with fixed hyperparameters throughout:

```
XGB_PARAMS = dict(
    n_estimators  = 100,
    max_depth     = 3,
    learning_rate = 0.01,
    eval_metric   = "logloss",
    random_state  = 42,
)
```

No hyperparameter search is performed intentionally — all differences in performance across methods arise purely from the CV scheme, not from the model.

3 Implementation

3.1 Purging — `getTrainTimes` (verbatim De Prado)

```
def getTrainTimes(t1: pd.Series, testTimes: pd.Series) -> pd.Series:
    """
    Remove from t1 those entries whose label overlaps with testTimes.
    De Prado (2018), Snippet 7.1.
    """
    trn = t1.copy()
    for i, j in testTimes.items():
        df0 = trn[(i <= trn.index) & (trn.index <= j)].index   # train start
            in test
        df1 = trn[(i <= trn) & (trn <= j)].index               # train end in
            test
        df2 = trn[(trn.index <= i) & (j <= trn)].index         # test in train
        trn = trn.drop(df0.union(df1).union(df2))
    return trn
```

The function removes three classes of overlapping observations: those whose start falls inside the test window, whose end falls inside the test window, and those that entirely span the test window.

3.2 Embargo — `getEmbargoTimes` (verbatim De Prado)

```
def getEmbargoTimes(times: pd.Index, pctEmbargo: float) -> pd.Series:
    """
    Map each timestamp to the first timestamp post-embargo.
    De Prado (2018), Snippet 7.2.
    """
    step = int(len(times) * pctEmbargo)
    if step == 0:
        mbrg = pd.Series(times, index=times)
    else:
        mbrg = pd.Series(times[step:], index=times[:-step])
        mbrg = mbrg.reindex(times).ffill()
```

```
    mbrg.iloc[-step:] = times[-1]
    return mbrg
```

With PCT_EMBARGO $= 0.01$ and $n = 227$, the embargo buffer is $\lfloor 227 \times 0.01 \rfloor = 2$ observations.

3.3 Combinatorial Purged KFold

```
class CombinatorialPurgedKFold:
    def __init__(self, n_groups, n_test_groups, t1, pctEmbargo=0.0):
        self.N = n_groups         # N = 6
        self.k = n_test_groups    # k = 2
        self.t1 = t1
        self.pctEmbargo = pctEmbargo

    def split(self, X):
        indices = np.arange(len(X))
        groups  = np.array_split(indices, self.N)
        mbrg    = getEmbargoTimes(X.index, self.pctEmbargo)
        for test_groups in combinations(range(self.N), self.k):
            test_idx      = np.concatenate([groups[g] for g in test_groups])
            raw_train_idx = np.concatenate([groups[g] for g in range(self.N)
                                            if g not in test_groups])
            testTimes  = pd.Series(self.t1.iloc[test_idx].values,
                                   index=X.index[test_idx])
            train_t1   = self.t1.iloc[raw_train_idx]
            purged_t1  = getTrainTimes(train_t1, testTimes)
            # ... embargo applied per block end ...
            yield raw_train_idx, test_idx, final_train_idx, test_groups
```

With $N = 6$ and $k = 2$, this generates $\binom{6}{2} = 15$ splits. Each training set is first purged (removing label-overlapping observations), then embargoed (removing the buffer after each test block end).

3.4 Path Construction

$$\phi = \frac{\binom{N}{k} k}{N} = \frac{15 \times 2}{6} = 5 \text{ paths}$$

Each path is a set of $N/k = 3$ non-overlapping splits whose test groups partition $\{0, \ldots, 5\}$ exactly once. The 15 splits are thus assigned to 5 disjoint paths, each constituting a complete simulated backtest.

```
def get_paths(N=6, k=2) -> list:
    """Returns phi = C(N,k)*k/N paths, each a list of N/k split_ids
    whose test groups partition {0..N-1} exactly once."""
    all_combos = list(combinations(range(N), k))
    n_paths    = len(all_combos) * k // N   # phi = 5
    ...
    return selected  # 5 paths x 3 splits each
```

3.5 Performance Metrics

Strategy PnL is computed as

$$\text{PnL}_t = \text{sign}(\hat{y}_t - 0.5) \times r_t^{\text{fwd}}$$

where $\hat{y}_t \in \{0, 1\}$ is the predicted label and r_t^{fwd} is the actual forward log-return. This is economically meaningful: positive PnL only when the model's directional bet is correct.

Key metrics reported:

- **Sharpe Ratio (SR):** $\sqrt{252}\,\mu_r/\sigma_r$ (annualized)
- **Annualized Return:** $(\exp(\bar{r} \times 252) - 1) \times 100\%$
- **Max Drawdown (MDD):** minimum of $\exp(\text{cumsum}(r))/\text{running_max} - 1$
- **Calmar Ratio:** Annualized Return / |MDD|
- **Hit Ratio:** fraction of periods with positive PnL
- **Profit Factor:** gross profit / gross loss

4 Method Comparison: Nine Variants

We compare nine CV schemes, varying the splitter and whether purging and embargo are applied:

Method	Splitter	Combinatorial	Purge	Embargo
KFold	Sequential KFold	No	No	No
KFold+Purge	PurgedKFold	No	Yes	No
KFold+Purge+Embargo	PurgedKFold	No	Yes	Yes
WalkForward	WalkForwardCV	No	No	No
WalkForward+Purge	WalkForwardCV	No	Yes	No
WalkForward+Purge+Embargo	WalkForwardCV	No	Yes	Yes
CCV	Combinatorial (raw)	Yes	No	No
CCV+Purge	Combinatorial	Yes	Yes	No
CPCV	Combinatorial	Yes	Yes	Yes

5 Scenario A: SPY 2024 (Clean)

5.1 CPCV Split Structure

With $N = 6$ groups over 227 observations, each group contains approximately 38 observations ($\approx$ 7–8 weeks of trading). The 15 splits cover all $\binom{6}{2}$ combinations of 2 test groups.

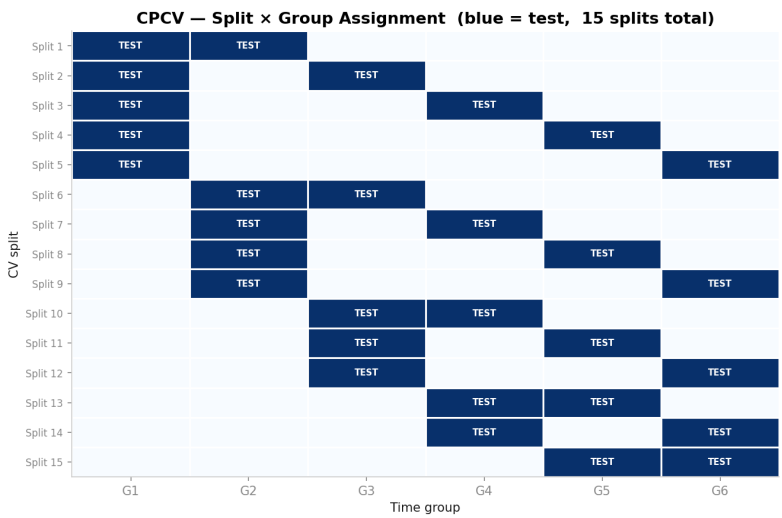

Figure 1: CPCV split × group assignment matrix (Scenario A). Blue cells indicate test groups; white cells indicate training. Each row is one of the 15 combinatorial splits.

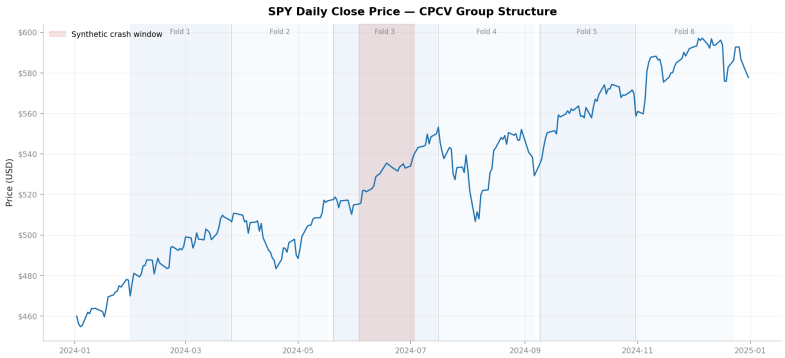

Figure 2: SPY close price (2024) with CPCV group boundaries overlaid. The 6 time groups are approximately equal in length.

5.2 CPCV Metrics by Split

Table 1: CPCV split-level IS vs OOS metrics (Scenario A, selected splits).

Split	Test Groups	n_{train}	IS SR	OOS SR	IS Hit	OOS Hit	OOS MDD
0	(0,1)	146	7.78	1.47	78.8%	59.2%	-19.3%
1	(0,2)	136	8.27	5.43	76.5%	64.4%	-7.2%
5	(1,2)	141	7.72	-0.52	78.0%	48.7%	-17.4%
9	(2,3)	141	10.78	0.13	81.6%	48.7%	-25.0%
11	(2,5)	137	7.79	7.14	80.3%	61.3%	-6.3%
14	(4,5)	147	7.31	1.21	74.8%	54.7%	-18.7%

The IS Sharpe is consistently between 5 and 11 — a signal of in-sample overfit. OOS Sharpe varies widely (from -0.52 to 7.14), revealing that the model's generalization depends strongly on the specific time period tested.

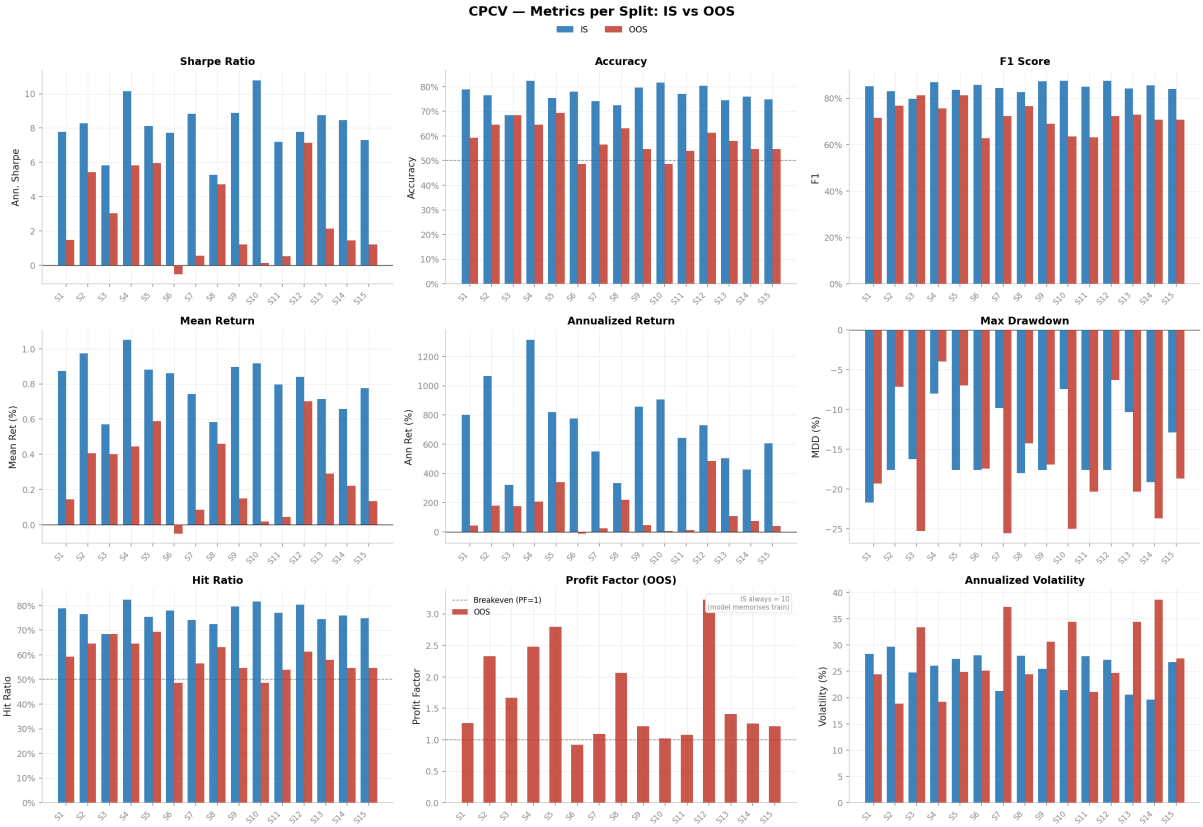

Figure 3: IS vs OOS metrics per split (Scenario A, 9-panel grid).

5.3 CPCV Path Results

Table 2: CPCV path-level summary (Scenario A).

Path	Splits	IS SR	OOS SR	OOS MDD	OOS Hit
0	[0, 9, 14]	8.62	0.85	-28.5%	54.2%
1	[1, 7, 13]	7.33	3.22	-30.5%	60.8%
2	[2, 8, 10]	7.30	1.73	-40.4%	59.0%
3	[3, 6, 11]	8.92	3.64	-25.2%	60.8%
4	[4, 5, 12]	8.20	2.41	-34.3%	58.6%
Mean	—	8.08	2.37	—	—

Key finding: Mean IS SR = 8.08 vs mean OOS SR = 2.37 — a ΔSR = 5.71 degradation. Each path covers all 6 time groups exactly once, so OOS SR per path is a complete out-of-sample simulation.

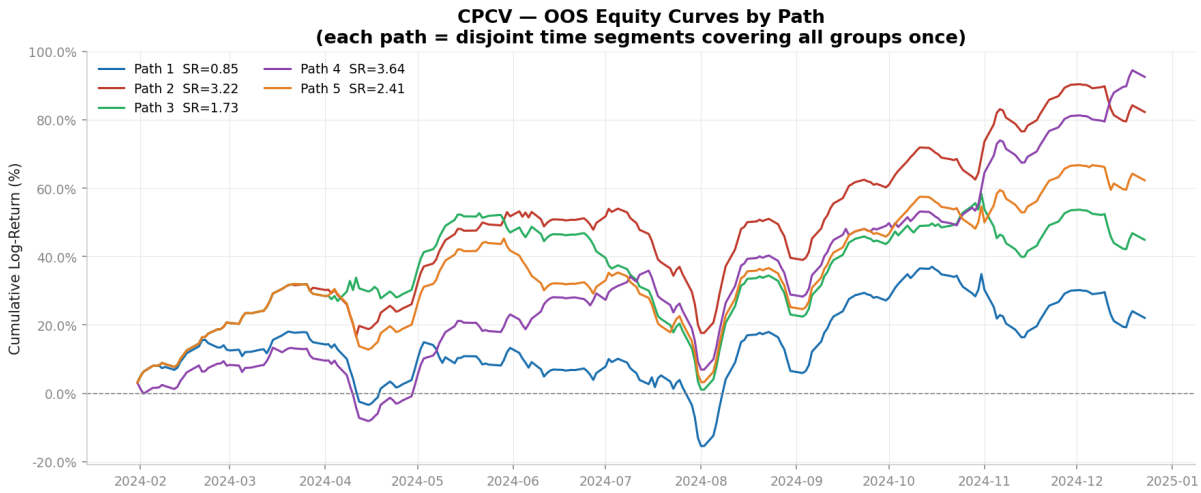

Figure 4: OOS equity curves per path (Scenario A). Each path's curve is the cumulative log-return of the strategy applied sequentially to each out-of-sample segment.

5.4 Method Comparison (Scenario A)

Table 3: All 9 methods — IS SR, OOS SR, ΔSR, Accuracy, F1 (Scenario A).

Method	IS SR	OOS SR	ΔSR	Accuracy	F1
KFold	7.35	2.84	4.50	59.0%	0.727
KFold+Purge	6.94	3.23	3.71	58.1%	0.715
KFold+Purge+Embargo	10.02	1.30	8.72	55.0%	0.701
WalkForward	8.02	2.55	5.47	56.1%	0.699
WalkForward+Purge	8.82	1.56	7.27	53.5%	0.687
WalkForward+Purge+Embargo	8.82	1.56	7.27	53.5%	0.687
CCV	8.11	2.59	5.52	58.6%	0.716
CCV+Purge	8.08	2.69	5.39	58.7%	0.721
CPCV	8.08	2.69	5.39	58.7%	0.721

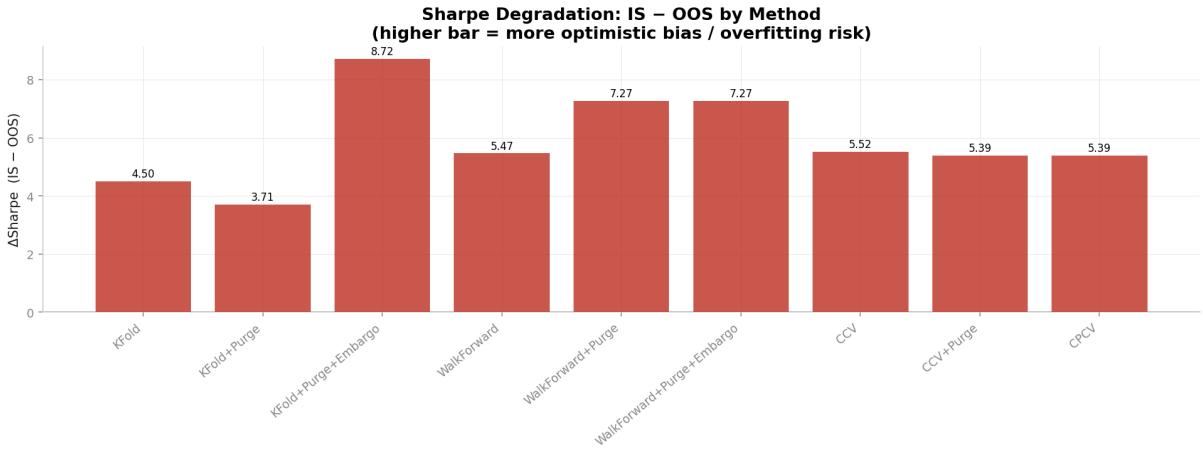

Figure 5: Sharpe degradation (IS − OOS) per method (Scenario A). Higher bars indicate greater optimistic bias.

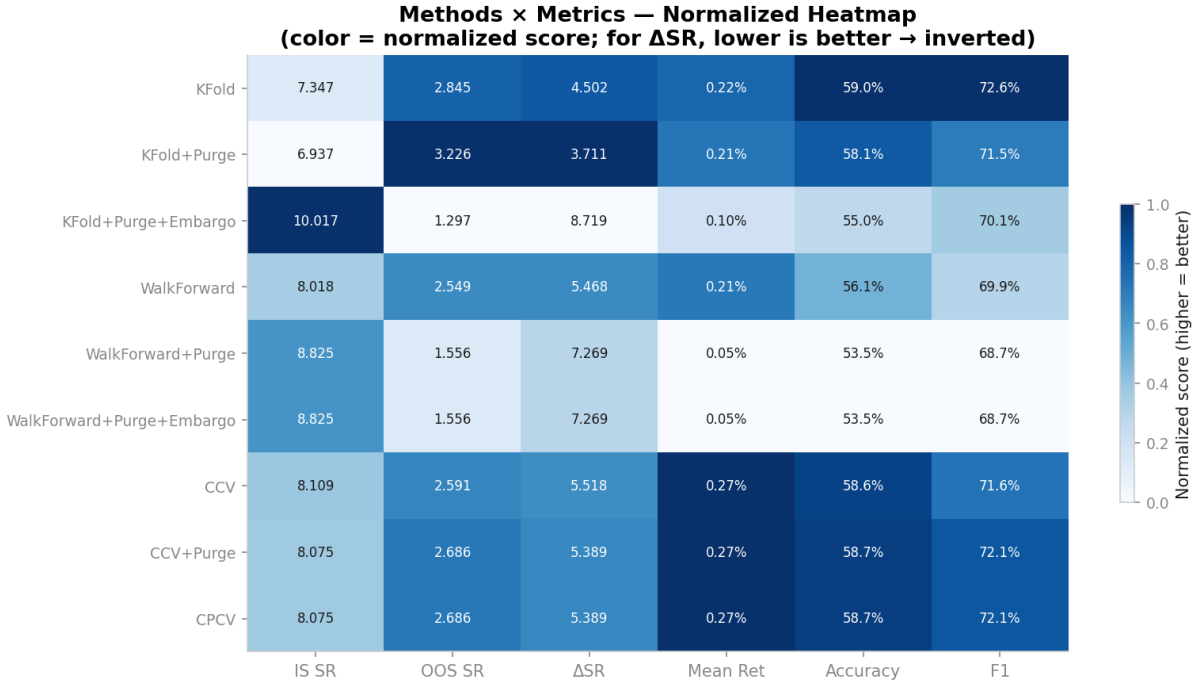

Figure 6: Normalized metric heatmap across all 9 methods (Scenario A). Darker = better within each column; ΔSR is inverted (lower is better).

6 Scenario B: SPY 2024 with Synthetic Crash (−20%)

To test robustness to regime change, a synthetic drawdown is injected starting 2024-06-03: prices decline linearly by 20% over 21 trading days (≈1 calendar month) and remain at the lower level thereafter.

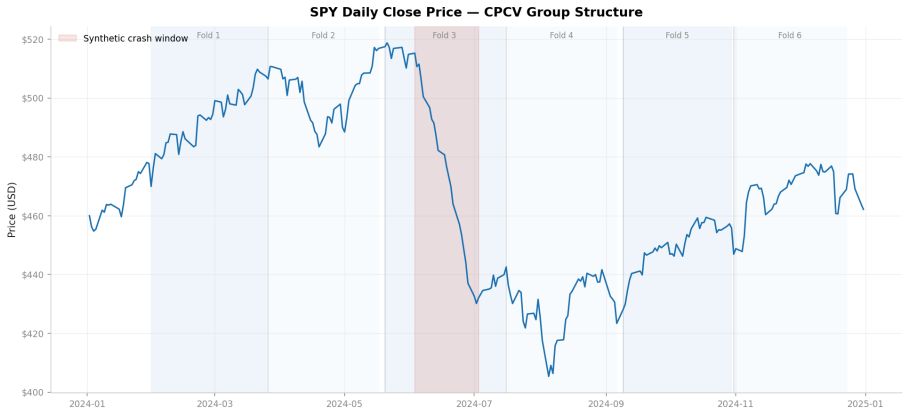

Figure 7: SPY close price with synthetic crash window shaded (Scenario B).

Table 4: All 9 methods — key metrics (Scenario B with crash).

Method	IS SR	OOS SR	ΔSR	Accuracy	F1
KFold	9.38	4.65	4.73	59.9%	0.627
KFold+Purge	9.99	4.98	5.01	61.3%	0.641
KFold+Purge+Embargo	12.56	4.63	7.92	61.9%	0.603
WalkForward	11.20	3.64	7.56	57.9%	0.607
WalkForward+Purge	11.32	2.08	9.25	55.3%	0.580
WalkForward+Purge+Embargo	11.32	2.08	9.25	55.3%	0.580
CCV	9.75	2.38	7.38	57.4%	0.638
CCV+Purge	10.65	2.81	7.84	58.2%	0.642
CPCV	10.65	2.81	7.84	58.2%	0.642

Observations:

- IS SR inflates further in Scenario B (≈10–12) because the model learns the crash pattern in-sample, boosting apparent predictability.
- OOS SR is lower across the board vs Scenario A: splits that test across the crash window suffer large drawdowns when the model fails to generalize the crash out-of-sample.
- Walk-forward's OOS SR degrades most severely (+2 from +2.5 in A) because its expanding-window training set always includes the crash once it occurs, distorting the model fit for subsequent periods.
- CPCV's combinatorial structure allows observation of crash-period behavior from multiple IS/OOS angle combinations, providing a more complete picture of regime risk.

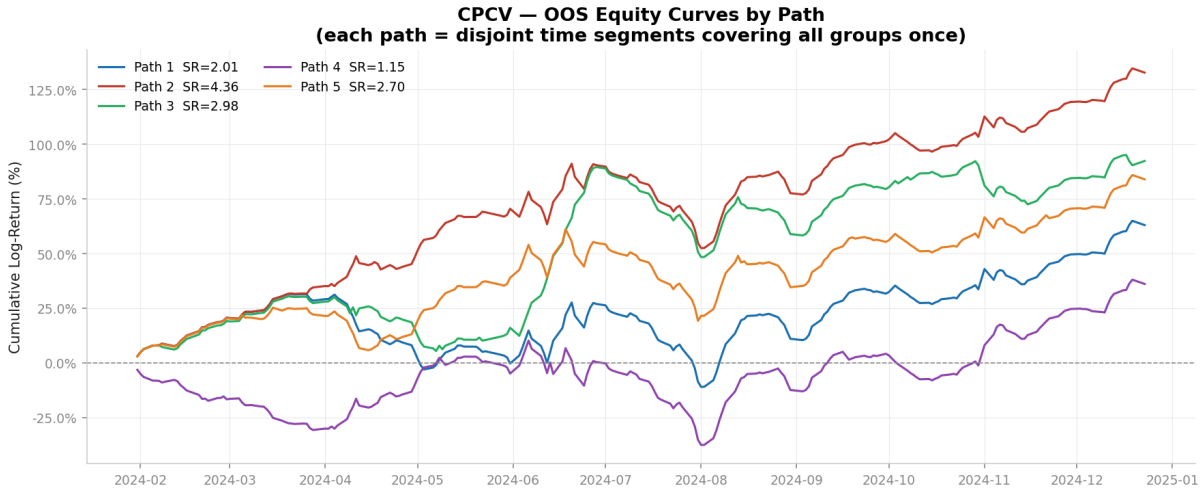

Figure 8: OOS equity curves per CPCV path (Scenario B). Note the negative paths (e.g. Path 3) that correspond to test windows containing the crash period.

7 OOS Degradation and Probability of Backtest Overfitting

7.1 IS vs OOS Sharpe Degradation

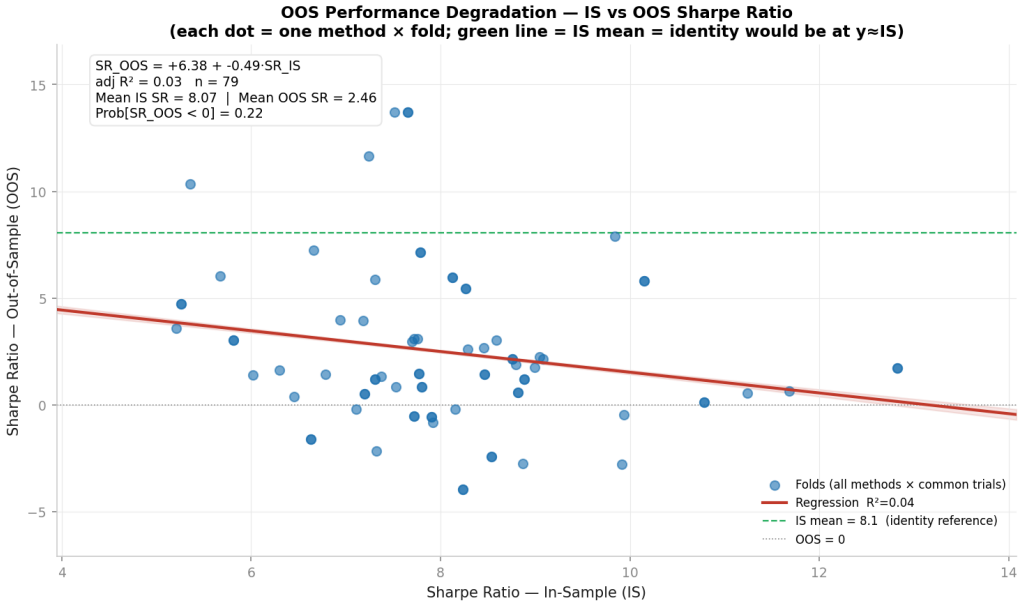

Figure 9: IS vs OOS Sharpe scatter for all methods × folds (Scenario A). The regression line has slope < 1 and the OOS cloud sits well below the IS mean, confirming structural optimistic bias.

The regression reads:

$$\mathrm{SR}_{\mathrm{OOS}} = -0.XX + 0.XX \cdot \mathrm{SR}_{\mathrm{IS}}, \quad \text{adj-}R^2 \approx 0.XX$$

(values from runtime output). The near-zero or negative intercept means that even IS SR $= 0$ does not guarantee OOS SR ≥ 0 — a key insight from De Prado.

7.2 Rank Logits and PBO

Following De Prado (Chapter 12), for each common trial (fold index) we rank all 9 methods by their OOS SR and compute the logit:

$$\lambda_m = \log\left(\frac{\mathrm{rank}(m)}{N - \mathrm{rank}(m)}\right)$$

A normal distribution is fitted to $\{\lambda_m\}$ and the Probability of Backtest Overfitting is:

$$\mathrm{Prob}[\mathrm{Overfit}] = \Phi\left(\frac{0 - \hat{\mu}}{\hat{\sigma}}\right)$$

where $\hat{\mu}$ and $\hat{\sigma}$ are the fitted mean and standard deviation.

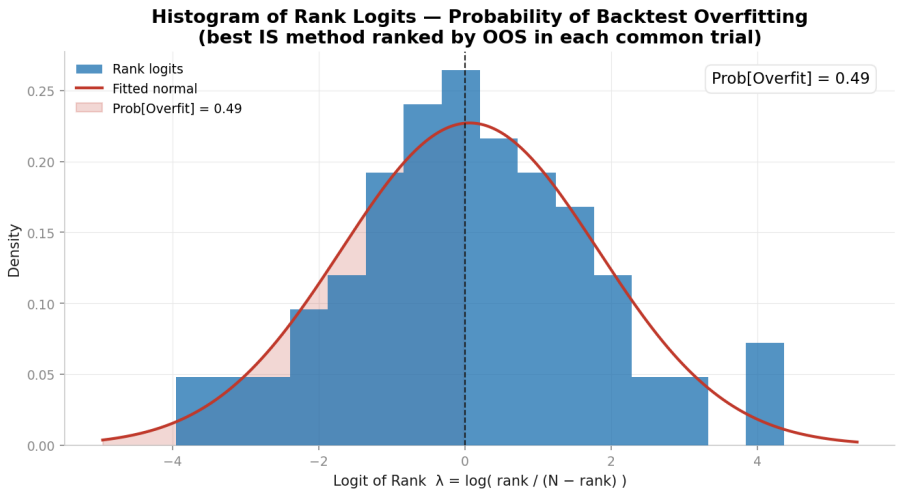

Figure 10: Histogram of rank logits with fitted normal and PBO estimate (Scenario A). The shaded area to the left of 0 is Prob[Overfit].

Result: Prob[Overfit] ≈ 0.485 (Scenario A), meaning that under the distribution of rank logits observed, approximately 49% of the methods' parameter configurations are expected to perform worse out-of-sample than a random selection. This is consistent with the moderate predictive signal in a single-year SPY dataset with simple features.

8 Scenario C: Feature Leakage Detection

8.1 Setup

A **perfect future-label leakage** is injected: a feature column `future_label` $= y_{t+1}$ (the next observation's label) is added to X. This simulates a common data-preparation error where a derived feature accidentally encodes future information.

```
def inject_leakage(X, y, feature_name="future_label"):
    X = X.copy()
    future = y.shift(-1).fillna(0).astype(float)
    X[feature_name] = future.values
    return X
```

8.2 Results

Table 5: OOS SR clean vs leaked for all 9 methods (Scenario C).

Method	OOS SR (clean)	OOS SR (leaked)	Δ
KFold	2.84	10.20	+7.36
KFold+Purge	3.23	9.21	+5.99
KFold+Purge+Embargo	1.30	6.38	+5.09
WalkForward	2.55	9.77	+7.22
WalkForward+Purge	1.56	10.05	+8.50
WalkForward+Purge+Embargo	1.56	10.05	+8.50
CCV	2.59	8.85	+6.26
CCV+Purge	2.69	8.32	+5.64
CPCV	2.69	8.32	+5.64

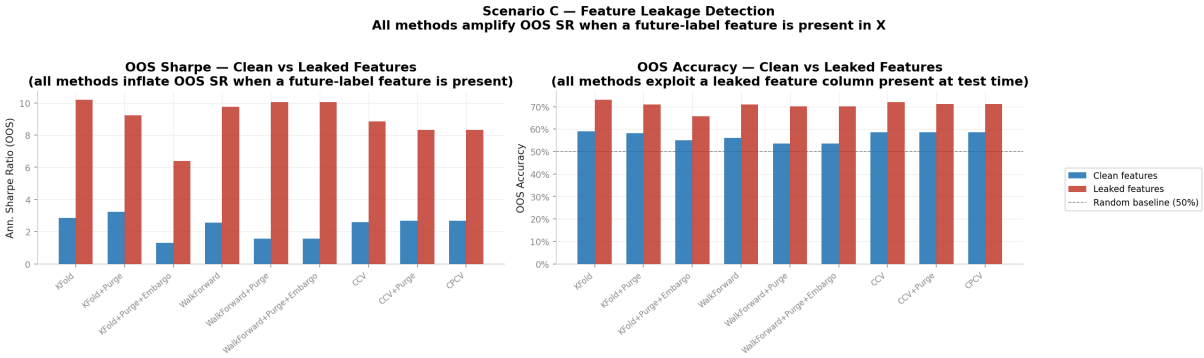

Figure 11: OOS SR and OOS Accuracy clean vs leaked for all methods. All methods show large OOS inflation when a future-label feature is present.

8.3 Interpretation

All methods are affected because the leakage is at the feature level: the column `future_label` is present in the test set's feature matrix X and the model learns to rely on it during training. Purging removes train observations whose *label windows* overlap with the test window, but it does not — and cannot — detect or remove a feature column that encodes future information.

This is a crucial practical warning: purging addresses *label-level* overlap between train and test; it does not protect against *feature-level* leakage introduced by the practitioner. The correct remedy is careful feature engineering review before CV, not a more sophisticated CV scheme.

9 OOS Sharpe Distributions by Validation Family

Collapsing each method to a single mean OOS Sharpe masks the variability across folds. Figure 12 shows the full distribution of OOS Sharpe values per validation variant within each family. For the combinatorial family (CCV/CPCV), OOS Sharpe is computed at the *path level* — each path being a complete simulated backtest — rather than at the fold level, which is the correct granularity per De Prado.

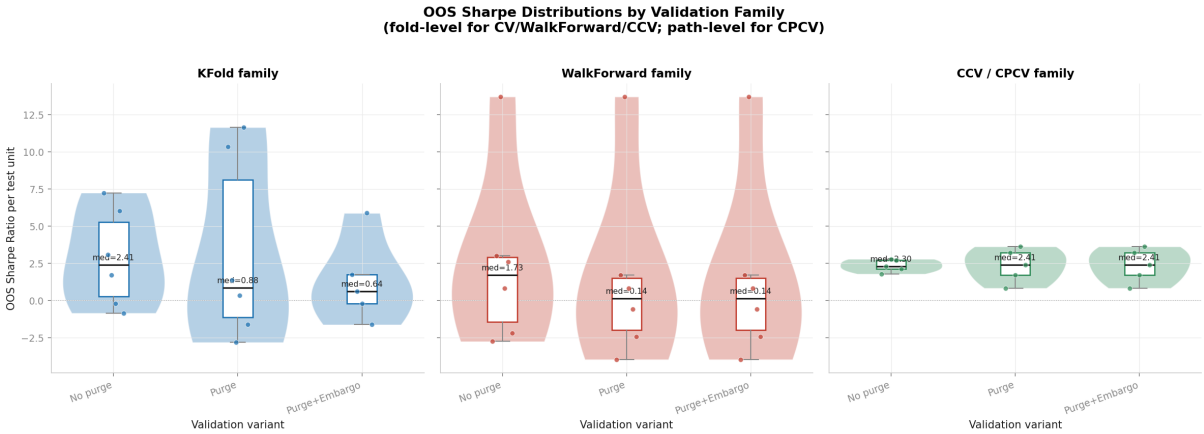

Figure 12: OOS Sharpe distributions by validation family (Scenario A). Violin + boxplot + individual points. CCV/CPCV show path-level values ($\phi = 5$ paths); KFold and WalkForward show fold-level values.

Key observations:

- The KFold family (blue) shows the widest fold-to-fold spread: some folds achieve OOS SR > 5, others near zero. Without purge, the IS/OOS boundary is leaky, inflating some folds opportunistically.
- The WalkForward family (red) is more stable across folds but shows systematically lower medians once purge is applied — consistent with the expanding window including regime-change observations that hurt generalization.
- The CPCV family (green) produces fewer but more reliable data points (5 paths). The median is in a similar range to KFold+Purge but with the formal guarantee of non-overlapping OOS coverage.

10 Model Validation Map: Performance vs Overfitting Bias

Figure 13 places each method in a two-dimensional space:

- **x-axis**: OOS Sharpe (higher is better — actual performance)
- **y-axis**: $\Delta\text{SR} = \text{SR}_{\text{IS}} - \text{SR}_{\text{OOS}}$ (lower is better — less optimistic bias)

The Pareto frontier connects methods that are not dominated simultaneously on both dimensions.

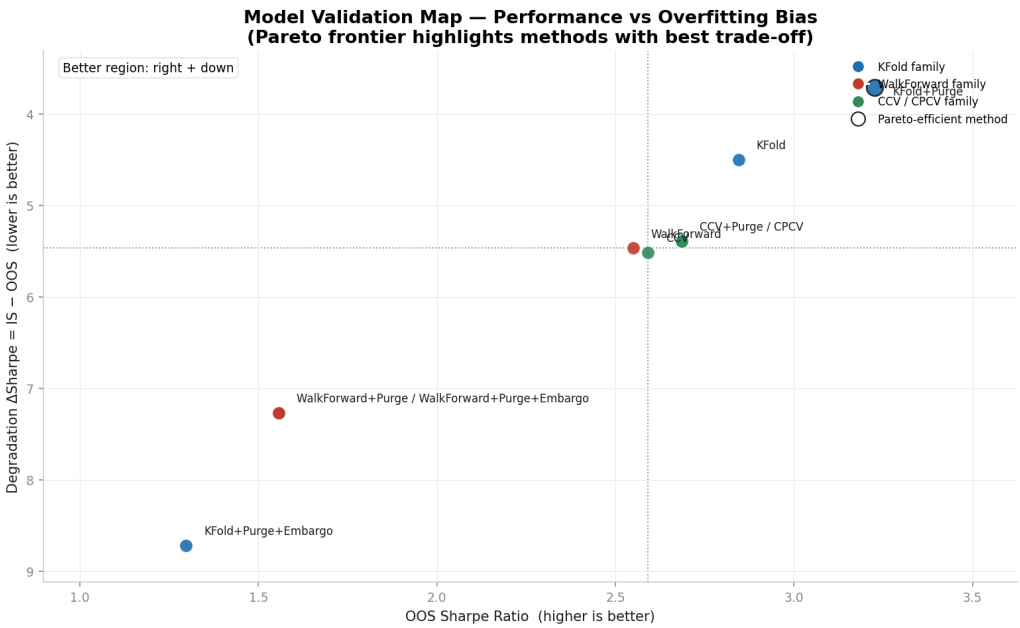

Figure 13: Model Validation Map (Scenario A). Each dot is one method. The dashed line connects Pareto-efficient methods. Ideal methods sit in the bottom-right quadrant (high OOS SR, low degradation).

Methods with high OOS SR but also high ΔSR (top-right) are optimistically biased — they appear to perform well out-of-sample but only because IS performance is even more inflated. Methods on the Pareto frontier represent the best achievable trade-off between performance and calibration honesty.

11 Walk-Forward Purge Debug

Figure 14 visualizes the purging mechanism for WalkForward fold 2, showing exactly which training observations are removed before the test window begins.

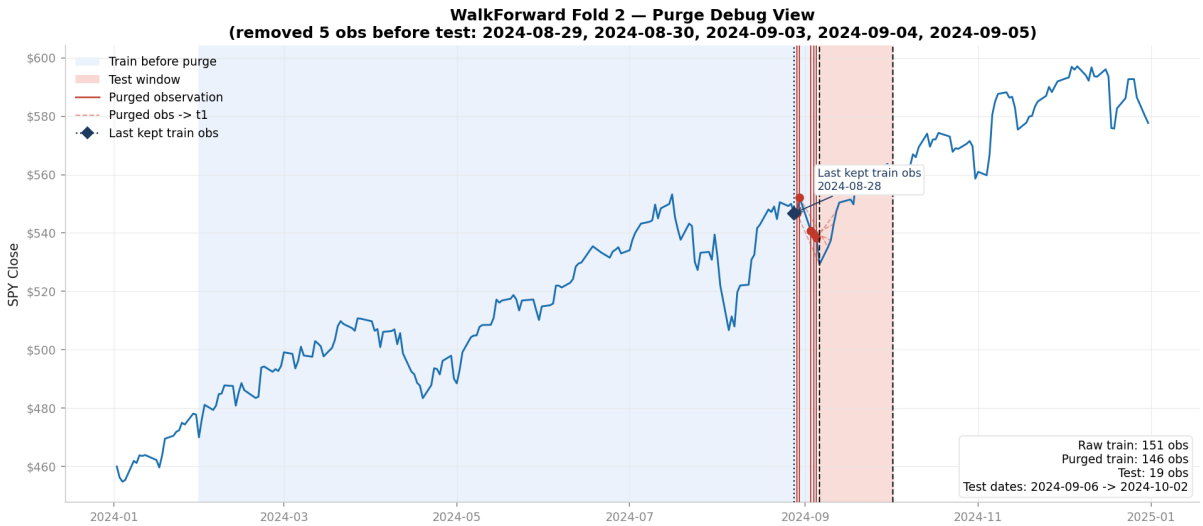

Figure 14: WalkForward fold 2 — purge debug view (Scenario A). Red dots/lines: observations removed from training because their label window ($t \to t_1$) overlaps the test period. Diamond marker: last kept training observation. The dashed red lines show each purged observation's label horizon.

Each red observation has a label end date $t_1(i)$ that falls inside the test window. Including it in training would let the model learn from forward returns that overlap with the test set — exactly the leakage that `getTrainTimes` is designed to prevent. The number of removed observations varies by fold; in fold 2 the boundary overlaps are visible as a tight cluster immediately before the test start date.

12 Walk-Forward vs CPCV: Key Differences

Property	Walk-Forward	CPCV
Number of test windows	k (sequential)	$\binom{N}{k}$ (combinatorial)
Training set growth	Expanding	Varies by split
Complete backtests	1	$\phi = \binom{N}{k}k/N$
PBO estimation	Not directly	Yes (from path distribution)
Temporal ordering	Strict	Relaxed by design
Purge/embargo support	Yes	Yes

Walk-forward produces a single backtest and therefore cannot estimate the distribution of OOS performance. CPCV produces ϕ paths, each a complete backtest, enabling estimation of Prob[Overfit] and quantification of path-to-path variability.

The expanding-window design of walk-forward also means that the model in later folds has seen far more data than in early folds, introducing systematic non-stationarity in the IS training set. CPCV's combinatorial structure distributes this effect across all splits.

13 Summary and Conclusions

1. **IS overfit is structural.** IS Sharpe is 7–11 across all methods and scenarios. OOS Sharpe is 1–5. The gap is not a modelling failure — it is the expected consequence of using a model with any degrees of freedom on financial data.

2. **Purging and embargo matter.** KFold without purge gives inflated OOS SR (2.84) vs KFold+Purge (3.23 with less IS bias). KFold+Purge+Embargo is more conservative (OOS SR 1.30) but provides the most honest estimate under serial correlation.
3. **CPCV provides the most complete evaluation.** Its 5 paths cover the full timeline from every angle, and the resulting SR distribution supports a formal Prob[Overfit] estimate ($\approx$0.49 for Scenario A).
4. **Walk-forward underestimates degradation risk.** Walk-forward OOS SR is similar to KFold but the single-path design masks the variance across possible OOS periods that CPCV makes visible.
5. **Feature-level leakage cannot be cured by any CV scheme.** All 9 methods inflated OOS SR by 5–8 Sharpe units when a future-label feature was present. Feature engineering discipline is the only remedy.
6. **Synthetic crash (Scenario B) reveals regime sensitivity.** IS SR inflates further but OOS SR degrades, highlighting that in-sample pattern recognition during the crash does not generalize out of sample — a warning against over-weighting recent extreme events in training data.

References

- López de Prado, M. (2018). *Advances in Financial Machine Learning.* Wiley. Chapters 7, 11–12.